ML Foundations

Course Title

Instructor Name

March 24, 2026

Outline

Concepts in Machine Learning

Four ingredients:

Data	some input to a task we want an AI to do
Model	gets data, outputs something (a classification, an action, a prediction)
Loss function	quantifies how well the model did
Optimizer	updates the model to do the task better

The Training Loop in PyTorch

```
for x, y in dataloader:
    optimizer.zero_grad()
    prediction = model(x)          # forward pass
    loss = loss_fn(prediction, y) # compute loss
    loss.backward()                # backward pass
    optimizer.step()               # update parameters
```

This is *the* loop. Everything else in ML is about making each piece better.

Terminology

Parameters / weights the numbers that represent the model itself, updated by the optimizer (θ)

Activations the numbers created as we feed data through the network

Hyperparameters choices made *before* training: learning rate, batch size, architecture

Stochasticity and Batching

We want to minimize the true loss over the full data distribution:

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\ell(f_\theta(x), y) \right]$$

But we only ever see a finite **batch** $\mathcal{B} \subset \mathcal{D}$ at each step:

$$\hat{\mathcal{L}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{(x,y) \in \mathcal{B}} \ell(f_\theta(x), y)$$

- Every gradient is a noisy estimate of the true gradient
- Batch size controls the noise level

Tensors

The central object in PyTorch: multidimensional arrays of numbers.

We use tensors to represent: data, parameters, activations, outputs, losses.

```
x = torch.tensor([1.0, 2.0, 3.0])
M = torch.zeros(3, 4)
```

By convention, everything except parameters has the **batch dimension first**, so we can process many samples at once.

Tensor Operations

```
torch.exp(x)            # elementwise exponential
A @ B                   # matrix multiply
x.unsqueeze(0)          # add a dimension
```

Einstein notation makes index operations explicit:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

```
torch.einsum('ik,kj->ij', A, B)
```

Einops

Readable notation for tensor reshaping:

```
# PyTorch (what does this do?)
x.reshape(B, S, n_heads, d_head).permute(0, 2, 1, 3)

# Einops (self-documenting)
rearrange(x, 'b s (h d) -> b h s d', h=n_heads)
```

- `rearrange`: reshape, permute, split, merge
- `reduce`: pool over dimensions
- `repeat`: tile/broadcast

Autograd and the Computation Graph

Tensors keep track of a computational graph as you operate on them.

1. Build up expressions from tensors with `requires_grad=True`
2. Call `.backward()` on a scalar result
3. Every tensor in the graph can now access its gradient via `.grad`

You never write derivatives by hand. The graph is rebuilt every forward pass ("define-by-run").

Devices

- Tensors live on a **device**: CPU or GPU (`cuda:0`)
- Move with `x.to('cuda')`
- All tensors in an operation must be on the same device
- GPU memory is the main bottleneck for model size and batch size

Exercise 1: PyTorch Basics

25 minutes

Loss Functions: Supervised Learning

The loss function defines *what* we are optimizing.

When we can directly verify the output:

- Predict something continuous (position of a ball) $\Rightarrow$ MSE
- Predict something discrete (class, next word) $\Rightarrow$ **cross-entropy loss**

$$\mathcal{L} = -\sum_i y_i \log \hat{y}_i$$

Minimizing cross-entropy makes the model **well-calibrated**: its output numbers can be interpreted as probabilities.

Loss Functions: Human Preferences

We do not always have the "correct label." Sometimes we only know a pattern outputs should have.

Example: rating text quality, given only pairwise human preferences ("I prefer A over B").

The preferred text should be rated higher:

$$\mathcal{L} = -\log \sigma(r_\theta(x_w) - r_\theta(x_l))$$

[**christiano2023a**]

Loss Functions: Reinforcement Learning

Sometimes we cannot say what the correct output looks like, but we can identify one when we see it (e.g. winning at chess).

- RL loss functions are more complex
- Often involve a learned reward model (from the previous slide) combined with a policy objective

Train Loss vs. Test Loss

Underfitting	high train loss, high test loss (model too simple or undertrained)
Overfitting	low train loss, high test loss (model memorizes training data)
Good fit	both low (generalizes to unseen data)

Regularization (L1, L2, dropout) constrains the model to reduce overfitting.

Optimizers

We have a gradient. Which direction do we step?

Stochastic Gradient Descent

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}$$

- α: learning rate (hyperparameter)
- Simple and robust
- Can be slow: oscillates across narrow valleys, crawls along them

Momentum

$$v_{t+1} = \beta\, v_t + (1-\beta)\, \nabla_\theta \mathcal{L}$$

$$\theta_{t+1} = \theta_t - \alpha\, v_{t+1}$$

- Keeps a running velocity, like a ball rolling downhill
- Accelerates through flat regions
- Dampens oscillations in steep directions
- Can roll over small local minima

Adam = Momentum + RMSProp

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t \qquad v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t^2$$

$$\hat{m} = \frac{m_{t+1}}{1-\beta_1^{t+1}} \qquad \hat{v} = \frac{v_{t+1}}{1-\beta_2^{t+1}}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

- Adaptive per-parameter learning rates + momentum
- Default choice for most deep learning
- Typical: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Exercise 2: Implement Optimizers

25 minutes

The MLP

- Input $\rightarrow$ hidden layer $\rightarrow$ output
- Without a hidden layer, we can only represent linear functions
- The hidden layer with a nonlinearity lets us express richer functions

Activation Functions

Without nonlinearities, stacking linear layers just gives another linear layer.

- **ReLU**: $\max(0, x)$, simple and effective
- **GELU**: smooth approximation, used in transformers
- **Sigmoid/Tanh**: saturate at extremes, mostly historical

Universal Approximation

Theorem (Cybenko, Hornik et al.): A single hidden layer with nonlinear activations can approximate any continuous function on a compact set to arbitrary precision.

- Existence result, not a practical recipe
- Says nothing about how many neurons you need
- In practice, deeper is much more efficient

Depth: Stacking Layers

We can more easily compute things if we compute them in multiple steps.

- Example: classifying a dog is easier if early layers detect ears and snouts, later layers combine them into "head"
- Models have layers that mostly look the same but learn different things
- The deep learning recipe: repeat a simple block many times

Over/underparameterization:

- Too few parameters $\Rightarrow$ cannot represent the function
- Too many $\Rightarrow$ can memorize, but modern networks often still generalize

Residual Connections

Deep networks suffer from **vanishing gradients**: the chain rule often makes gradients go to zero through many layers.

Fix: add a skip connection so every layer can start learning something.

$$x_{l+1} = x_l + f(x_l)$$

- Gradient through the skip is exactly 1
- Network only needs to learn the *residual* $f(x_l)$
- Enables training networks with hundreds of layers

Architecture Encodes Symmetry

The right architecture exploits structure in the data.

Example: in an image, a dog ear looks the same regardless of position. We need a detector that is **translation invariant**, so we should not learn a separate detector for every location. An architecture that respects a symmetry needs fewer parameters and less data to learn the same function.

Convolutional Neural Networks

- A small kernel slides across the input (weight sharing)
- Same features detected regardless of position
- Hierarchy: edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects

Transformers (Preview)

Text has a similar symmetry: each word is similar, but understanding requires long-range connections, not just neighbors.

- **Self-attention**: each token attends to all other tokens
- Repeated blocks of: attention $\rightarrow$ feedforward $\rightarrow$ residual connection
- Dominant architecture for language (and increasingly vision, audio, ...)
- We will go much deeper in the afternoon session

Exercise 3: Build Simple Architectures

15 minutes

Hyperparameters

Any property not set by gradient descent is set by the experimenter:

- Model architecture (depth, width, . . .)
- Batch size, number of epochs
- Learning rate, momentum, and other optimizer settings

Finding Good Hyperparameters

- Some we have figured out once and reuse (e.g. $\beta_1 = 0.9$)
- Some we sweep over: run many times, pick the best (typically learning rate)
- Some follow **scaling laws**: cheap small experiments predict large-scale performance

Scaling Laws

$$L(N) \propto N^{-\alpha}$$

- Loss decreases as a power law with model size, data, or compute
- Enables predicting performance before training the full model
- Compute-optimal training: balance model size and data

Exercise 4: Hyperparameter Tuning

Until lunch

References I